DIDs

The AT Protocol uses Decentralized Identifiers (DIDs) as persistent, long-term account identifiers. DID is a W3C standard, and atproto deliberately supports only a small "blessed" subset of DID methods rather than all of them:

- `did:web`: a W3C standard method based on HTTPS (and DNS), where the identifier section is a hostname. It is supported as an independent alternative to `did:plc`, but it is tied to the domain name used, and offers no way to migrate or to recover from losing control of the domain. Only hostname-level `did:web` DIDs are supported in atproto (no path-based DIDs), and the same top-level-domain restrictions that apply to handles apply here too. `localhost` (with optional port) is allowed only in testing and development.
- `did:plc`: a novel DID method developed by Bluesky.

A small number of additional methods may be supported in the future, but the intention is explicitly *not* to support many DID methods. Implementations should still be flexible when they encounter unsupported methods, and should distinguish between "invalid DID syntax", "unsupported DID method", and "supported DID method, but specific DID resolution failed".

DID syntax

Generic DID syntax constraints usable for validation in atproto: the URI is ASCII-only (`A-Z a-z 0-9 ._:%-`), case-sensitive, starts with lowercase `did:`, has a method segment of one or more lowercase letters followed by `:`, may not end in `:`, and percent signs must always be followed by two hex characters. Query and fragment parts are not allowed in atproto. There is an initial hard length limit of 2048 characters (best practice is to keep DIDs under 64 characters).

The spec's starting-point regex:

```
// NOTE: does not constrain overall length
/^did:[a-z]+:[a-zA-Z0-9._:%-]*[a-zA-Z0-9._-]$/
```

The spec's examples. Valid DIDs for use in atproto (correct syntax, and supported method):

```
did:plc:z72i7hdynmk6r22z27h6tvur
did:web:blueskyweb.xyz
```

?

Valid DID syntax (would pass Lexicon syntax validation), but unsupported DID method:

```
did:method:val:two
did:m:v
did:method:::val
did:method:-:_:.
did:key:zQ3shZc2QzApp2oymGvQbzP8eKheVshBHbU4ZYjeXqwSKEn6N
```

Invalid DID identifier syntax (regardless of DID method):

```
did:METHOD:val
did:m123:val
DID:method:val
did:method:
did:method:val/two
did:method:val?two
did:method:val#two
```

DIDs are case-sensitive. The currently supported methods happen not to be case-sensitive and could be safely lowercased, but protocol implementations should reject DIDs with invalid casing. Case normalization is permissible on user-controlled input such as URL paths or text fields.

DID documents

Resolving a DID yields a DID document, from which atproto-specific information is extracted. The parsing is agnostic to the DID method used. Three things get pulled out: the current handle, the signing key, and the PDS location.

The handle comes from the `alsoKnownAs` array:

> The current **handle** for the DID is found in the `alsoKnownAs` array. Each element of this array is a URI. Handles will have the URI scheme `at://`, followed by the handle, with no path or other URI parts. The current primary handle is the first valid handle URI found in the ordered list. Any other handle URIs should be ignored.

AT Protocol specification, "AT Protocol DIDs"

The DID document alone proves nothing about the handle. Anyone can put any hostname in their own document:

> It is crucial to validate the handle bidirectionally, by resolving the handle to a DID and checking that it matches the current DID document.

AT Protocol specification, "AT Protocol DIDs"

The DID is the primary account identifier, so an account with no valid, confirmed handle can still in theory participate in the ecosystem. Software should either not display any handle for such an account, or clearly indicate that the associated handle is invalid.

The signing key sits in the `verificationMethod` array, in an object with `id` ending `#atproto`, `controller` matching the DID itself, and `type` matching the fixed string `"Multikey"`. The first valid atproto signing key is used; others are ignored. The `publicKeyMultibase` field holds the public key in multibase encoding. This key signs repository commits (see the Repository section below). An account with no valid signing key in its DID document is broken.

The PDS service location sits in the `service` array, with `id` ending `#atproto_pds` and `type` matching `AtprotoPersonalDataServer`. The first matching entry is used. The `serviceEndpoint` must be an HTTPS URL containing only scheme, hostname, and optional port, with no userinfo or path prefix. An account with no valid PDS location is broken, though a temporarily unreachable PDS (during migration or downtime, say) does not immediately make the account invalid.

Handles

DIDs are opaque and unfriendly for human use. Handles are a less-permanent account identifier layered on top. Because the handle-to-DID link is verified via DNS (and possibly HTTPS), every handle must be a valid network hostname.

Handle syntax

Summary of the syntax rules (Lexicon string format `handle`):

- ASCII only, at most 253 characters overall
- Split into segments ("labels") separated by ASCII periods; at least two segments required, so bare top-level domains are not allowed, and neither is "trailing dot" syntax
- Each segment: 1 to 63 characters, from ASCII letters (`a-z`), digits (`0-9`), and hyphens; segments cannot start or end with a hyphen
- The last segment (the TLD) cannot start with a digit
- Handles are not case-sensitive, and should be normalized to lowercase

Reference regex:

```
/^([a-zA-Z0-9]([a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?\.)+[a-zA-Z]([a-zA-Z0-9-]{0,61}[a-zA-Z0-
```

The spec's identifier examples. Syntactically valid handles (which may or may not have existing TLDs):

```
jay.bsky.social
8.cn
name.t--t        // not a real TLD, but syntax ok
XX.LCS.MIT.EDU
a.co
xn--notarealidn.com
xn--fiqa61au8b7zsevnm8ak20mc4a87e.xn--fiqs8s
```

```
xn--ls8h.test
example.t        // not a real TLD, but syntax ok
```

Invalid syntax:

```
jo@hn.test
💩.test
john..test
xn--bcher-.tld
john.0
cn.8
www.masełkowski.pl.com
org
name.org.
```

Beyond syntax, certain "reserved" TLDs must fail any attempt at registration or resolution even though they pass syntax validation: `.alt`, `.arpa`, `.example`, `.internal`, `.invalid`, `.local`, `.localhost`, `.onion`. The `.test` TLD may be used in development but should fail in real-world environments. The special value `handle.invalid` is used in API responses to indicate that there is no bi-directionally valid handle for a given DID; it cannot be used in most other contexts.

Handle resolution

Handles need to be resolved to a DID in almost all situations. There are two supported mechanisms. The DNS TXT method is the recommended and preferred one for individual handle configuration, but services should fully support both; the HTTPS method exists mainly for large web services that cannot easily automate millions of DNS records. Clients can also rely on a network service (their PDS, for example) to resolve for them via `com.atproto.identity.resolveHandle`.

The DNS TXT method registers a TXT record on the `_atproto` sub-domain under the handle hostname, with a value prefixed `did=` followed by the full DID.

```
For example, the handle bsky.app would have a TXT record on the name
_atproto.bsky.app, and the value would look like
did=did:plc:z72i7hdynmk6r22z27h6tvur.
```

TXT values not starting with `did=` are ignored. Only a single valid record should exist; if multiple valid records with different DIDs are present, resolution should fail (retry later or via a recursive resolver). DNSSEC is not required. This method cannot serve very long handles: prepending `_atproto.` can push past the 253-character DNS limit, so in practice handles should stay at or under 244 characters.

The HTTPS well-known method has a web server at the handle domain serve `/.well-known/atproto-did`, responding with a 2xx status, `Content-Type: text/plain`, and the DID as the

body with no prefix or wrapper. The spec's example for `bsky.app` (a GET to `https://bsky.app/.well-known/atproto-did`):

```
HTTP/1.1 200 OK
Content-Length: 33
Content-Type: text/plain
Date: Wed, 14 Jun 2023 00:47:21 GMT

did:plc:z72i7hdynmk6r22z27h6tvur
```

Secure HTTPS on the default port (443) is required for real-world resolutions; HTTP is only for local development. Redirects are allowed up to a reasonable number of hops, and clients should strip small amounts of surrounding whitespace from the body.

Bidirectional verification and conflicts

Resolution in one direction is never enough:

> Handles should not be trusted or considered valid until the DID is also resolved and the current DID document is confirmed to link back to the handle. The link between handle and DID must be confirmed bidirectionally, otherwise anybody could create handle aliases for third-party accounts.

AT Protocol specification, "Handle"

The two resolution methods may be run concurrently, and the spec defines what to do when they disagree:

> It is ok to attempt both resolution methods in parallel, and to use the first successful result available. If the two methods return conflicting results (aka, different DIDs), the DNS TXT result should be preferred, though it is also acceptable to record the result as ambiguous and try again later.

AT Protocol specification, "Handle"

Services should cache resolution results internally (DNS TTLs are usually too aggressive a lifetime for this use case) and re-resolve periodically. Handles are normalized to lowercase everywhere: stored, transmitted, and displayed. UIs may prefix a handle with `@`, but that is not valid handle syntax in records or APIs, and handles should not be truncated to local context. `@jay.bsky.social` should never be shown as `@jay`.

NSIDs

Namespaced Identifiers (NSIDs) are used to reference Lexicon schemas for records, XRPC endpoints, and more.

> The basic structure and semantics of an NSID are a fully-qualified hostname in Reverse Domain-Name Order, followed by a simple name. The hostname part is the **domain authority,** and the final segment is the **name**.

AT Protocol specification, "Namespaced Identifiers (NSIDs)"

For example, `com.example.fooBar` is a syntactically valid NSID, where `com.example` is the domain authority and `fooBar` is the name segment. The domain authority must be a valid handle with the segment order reversed; the name is an ASCII camel-case string.

Syntax rules:

- Overall: ASCII only, segments separated by periods, at least 3 segments, at most 317 characters total
- Domain authority: at most 253 characters, at least two segments; each segment 1 to 63 characters of letters, digits, and hyphens (no leading/trailing hyphen); the first segment (the TLD) cannot start with a digit; *not case-sensitive*, and should be normalized to lowercase
- Name: 1 to 63 characters, ASCII letters and digits only (no hyphens), cannot start with a digit; *case-sensitive* and should not be normalized

So the two halves of one identifier follow different case rules: `com.example` in `com.example.fooBar` can be lowercased like any hostname, but `fooBar` must be preserved exactly. The overall NSID is case-sensitive for display, storage, and validation. Publishing multiple NSIDs that differ only by casing is not allowed (namespace authorities are responsible for preventing that confusion), and implementations should not force-lowercase NSIDs.

Reference regex:

```
/^[a-zA-Z]([a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(\.[a-zA-Z0-9]([a-zA-Z0-9-]{0,61}[a-zA-Z0-9]
```

The spec's examples. Syntactically valid NSIDs:

```
com.example.fooBar
net.users.bob.ping
a-0.b-1.c
a.b.c
com.example.fooBarV2
cn.8.lex.stuff
```

Invalid NSIDs:

```
com.exa💩ple.thing
com.example
com.example.3
```

When referring to a group or pattern of NSIDs, a trailing `*` "glob" character is allowed (`com.atproto.*` matches any NSID under the `atproto.com` domain authority, including nested sub-authorities; a free-standing `*` matches everything). There may be only a single star, it must be the last character, and it must sit at a segment boundary. Also, using a subdomain segment such as `sync` in `com.atproto.sync.getHead` requires control of the full domain `sync.atproto.com`, not just `atproto.com`.

TIDs & Record Keys

TIDs

A TID ("timestamp identifier") is a compact string identifier based on an integer timestamp. TIDs are sortable, URL-safe, and useful as a "logical clock"; they are currently used in atproto as record keys and for repository commit "revision" numbers.

Structure of a TID:

- a 64-bit integer, big-endian byte ordering
- encoded as `base32-sortable`, meaning the character set `234567abcdefghijklmnopqrstuvwxyz`
- no padding characters, but all digits are always encoded, so the length is always 13 ASCII characters; the TID corresponding to integer zero is `2222222222222`

The layout of the 64-bit integer:

- The top bit is always 0
- The next 53 bits represent microseconds since the UNIX epoch (53 bits being the maximum safe integer precision of a 64-bit float, as used by Javascript)
- The final 10 bits are a random "clock identifier"

Generators pick a random clock identifier to avoid collisions, across worker instances of a PDS cluster for example, and must ensure the output stream is monotonically increasing and never repeats, even when multiple TIDs are generated in the same microsecond or during clock smear. Millisecond-precision clocks should pad the timestamp (multiply by 1000).

Do not mistake TIDs for globally unique identifiers:

> In the decentralized context of atproto, the global uniqueness of TIDs can not be guaranteed, and an antagonistic repo controller could trivially create records re-using known TIDs.

AT Protocol specification, "Timestamp Identifiers (TIDs)"

Syntax rules: always 13 characters, base32-sortable character set, and the first character must be one of `234567abcdefghij` (because the high bit of the integer is always zero). Early versions allowed hyphens; those are no longer valid. Reference regex:

```
/^[234567abcdefghij][234567abcdefghijklmnopqrstuvwxyz]{12}$/
```

The spec's examples. Syntactically valid TIDs:

```
3jzfcijpj2z2a
7777777777777
3zzzzzzzzzzzz
2222222222222
```

Invalid TIDs:

```
# not base32
3jzfcijpj2z21
0000000000000

# case-sensitive
3JZFCIJPJ2Z2A

# too long/short
3jzfcijpj2z2aa
3jzfcijpj2z2
222

# legacy dash syntax *not* supported (TTTT-TTT-TTTT-CC)
3jzf-cij-pj2z-2a

# high bit can't be set
zzzzzzzzzzzzz
kjzfcijpj2z2a
```

Record Keys

A **Record Key** (often shortened to `rkey`) names and references an individual record within the same collection of an atproto repository. It appears as a segment in AT URIs and in the repo MST path. Each record Lexicon schema declares which record key type its collection uses:

- `tid`: the most common scheme, using TID syntax such as `3jzfcijpj2z2a`. TIDs are usually generated from a local clock at record creation, with mechanisms to ensure they always increment and are not reused within the same collection of a given repository. A record's original creation timestamp can be *inferred* from a TID key, but keys are end-user-specifiable and should not be trusted. An early motivation for TIDs was loose temporal ordering of records, which improves storage efficiency of the repository data structure (MST).
- `nsid`: the record key must be a valid NSID.
- `literal:<value>`: used when there should be only a single record in the collection, with a fixed, well-known key. The most common value is `self` (declared as `literal:self`).

- `any`: any string meeting the general record key syntax. This can encode semantics in the name (a domain name, integer, or transformed AT URI), enabling de-duplication and known-URI lookups.

Baseline syntax constraints for all record keys (Lexicon string type `record-key`): restricted to ASCII alphanumeric plus `.-_:~`; 1 to 512 characters; the specific values `.` and `..` are not allowed; must be usable in a repo MST path and in a URI path component. Record keys are case-sensitive (though all-lowercase keys are a recommended practice). Best practice is to keep key paths under 80 characters.

The spec's examples. Valid record keys:

```
3jui7kd54zh2y
self
example.com
~1.2-3_
dHJ1ZQ
pre:fix
_
```

Invalid record keys:

```
alpha/beta
.
..
#extra
@handle
any space
any+space
number[3]
number(3)
"quote"
dHJ1ZQ==
```

If you build indexes or databases over atproto data, two of the spec's warnings apply to you directly. First:

> Implementations should not rely on global uniqueness of TIDs, and should not trust TID timestamps as actual record creation timestamps. Record Keys are "user-controlled data" and may be arbitrarily selected by hostile accounts.

AT Protocol specification, "Record Keys"

Second, the uniqueness scope of a record key. The same Record Key value may be used under multiple collections in one repository, so:

> The tuple of `(did, rkey)` is not unique; the tuple `(did, collection, rkey)` is unique.

AT Protocol specification, "Record Keys"

Reusing the same TID across collections in one repository usually indicates a relationship between the records (a "sidecar" extension record, for example). Most software should treat record keys as opaque strings. Decoding TID keys to a `uint64` for use as a database key is discouraged, since it is not resilient to key format changes.

AT URIs: putting the identifiers together

The AT URI scheme (`at://`) references individual records in a specific repository, identified by either DID or handle. It can also reference a collection within a repository, or an entire repository. The spec's example pair, both referencing the same record:

```
at://did:plc:44ybard66vv44zksje25o7dz/app.bsky.feed.post/3jwdwj2ctlk26
at://bnewbold.bsky.team/app.bsky.feed.post/3jwdwj2ctlk26
```

The restricted form currently used in Lexicons:

```
AT-URI        = "at://" AUTHORITY [ "/" COLLECTION [ "/" RKEY ] ]

AUTHORITY     = HANDLE | DID
COLLECTION    = NSID
RKEY          = RECORD-KEY
```

So a full record AT URI composes exactly the identifiers covered above: an identity (handle or DID) as authority, an NSID as the collection, and a record key. If you are used to web URLs, the authority part does not mean what you expect:

> A major semantic difference between AT URIs and common URL formats like `https://`, `ftp://`, or `wss://` is that the "authority" part of an AT URI does not indicate a network location for the indicated resource. Even when a handle is in the authority part, the hostname is only used for identity lookup, and is often not the ultimate host for repository content (aka, the handle hostname is often not the PDS host).

AT Protocol specification, "AT URI Scheme (at://)"

To actually fetch the record, you resolve the identity, read the PDS location from the DID document, and query that host.

AT URIs are also not durable or content-addressed by themselves. Handle-based AT URIs break if the user changes their handle (and could even point at a different repo if the handle is reused), and even a DID-based AT URI says nothing about record *contents*, which may change or be removed over time. The spec's guidance:

> When referencing records, especially from other repositories, best practice is to use a DID in the authority part, not a handle. For application display, a handle can be used as a more human-readable alternative. In HTML, it is permissible to *display* the handle version of an AT-URI and *link* (`href`) to the DID version.

AT Protocol specification, "AT URI Scheme (at://)"

> When a *strong* reference to another record is required, best practice is to use a CID hash in addition to the AT URI.

AT Protocol specification, "AT URI Scheme (at://)"

The CID pins the exact content; the AT URI locates which repository and collection holds it. Neither alone is a strong reference: an AT URI is not content-addressed, and a bare CID does not encode where the content lives.

The Data Model

Records and messages in atproto are stored, transmitted, encoded, and authenticated in a consistent way. The core data model supports both a binary representation (CBOR) and a textual one (JSON). The binary form is more than an optimization; it is the authoritative form for anything cryptographic:

> When data needs to be authenticated (signed), referenced (linked by content hash), or stored efficiently, it is encoded in Concise Binary Object Representation (CBOR). CBOR is an IETF standard roughly based on JSON. The specific normalized subset of CBOR used in the atproto data model is called **DAG-CBOR**.

AT Protocol specification, "Data Model"

The data model is inspired by IPLD (Interplanetary Linked Data), but atproto does not adopt IPLD's JSON conventions:

> IPLD specifies a normalized JSON encoding called **DAG-JSON,** but atproto uses a different set of conventions when encoding JSON data. The atproto JSON encoding is not designed to be byte-determinisitic, and the CBOR representation is used when data needs to be cryptographically signed or hashed.

AT Protocol specification, "Data Model"

Normalizations like key sorting are not required or enforced for atproto JSON: only DAG-CBOR is a byte-reproducible representation. Signatures are over exact bytes, so signing works by encoding as DAG-CBOR, hashing the bytes (SHA-256), and signing the hash.

Nodes, blocks, and links

Distinct pieces of data are called **nodes**; a node encoded in binary DAG-CBOR is a **block**. Nodes can reference each other by ordinary string URLs/URIs, or strongly by content hash, which is called a **link**. Linked nodes form higher-level structures such as Merkle trees and DAGs. Unlike URLs, hash links do not encode a network location; location must be inferred from protocol context. In exchange, links are "self-certifying": returned data can be verified against the link hash, so content can be redistributed and trusted even from untrusted parties.

Links are encoded as Content Identifiers (CIDs), which have both binary and string representations and include a metadata code indicating whether they link to a node (DAG-CBOR) or to arbitrary binary data.

Data types

Lexicon Type	IPLD Type	JSON	CBOR	Note
`null`	null	Null	Special Value (major 7)	
`boolean`	boolean	Boolean	Special Value (major 7)	
`integer`	integer	Number	Integer (majors 0,1)	signed, 64-bit
`string`	string	String	UTF-8 String (major 3)	Unicode, UTF-8
-	float	Number	Special (major 7)	not allowed in atproto
`bytes`	bytes	`$bytes` Object	Byte String (major 2)	
`cid-link`	link	`$link` Object	CID (tag 42)	CID
`array`	list	Array	Array (major 4)	
`object`	map	Object	Map (major 5)	keys are always strings
`blob`	-	`$type: blob` Object	`$type: blob` Map	

Floats are excluded entirely. Round-tripping floats through machine-native formats and re-encoding is not always consistent, and in a content-addressed system a byte encoding you can't reproduce is poison. If integers cannot substitute, encode floats as strings or bytes.

Integers are 64-bit signed, but best practice limits them to 53 bits of precision for Javascript compatibility.

Null, missing, and false-y are three different things. Explicitly setting a field to `null` differs semantically from omitting the field, and both differ from false-y values like `false`, `0`, empty lists, or empty objects.

Field names starting with `$` are reserved for the data model and protocol itself (`$bytes`, `$link`, `$type`). Implementations should ignore unknown `$` fields so the protocol can evolve; applications should not define new ones.

The `blob` type

References to "blobs" (arbitrary files, such as images) have a consistent format detectable without any Lexicon. A blob node is a map with:

- `$type` (string, required): fixed value `blob` (note: not a valid NSID)
- `ref` (link, required): CID reference to the blob, with multicodec type `raw`; in JSON, encoded as a `$link` object
- `mimeType` (string, required, not empty): content type, `application/octet-stream` if unknown
- `size` (integer, required, positive, non-zero): length in bytes

There is also a deprecated legacy blob format (a string `cid` field plus `mimeType`, with no `$type`) still found in some records in the wild. Implementations should not error on it, but must never write it.

JSON representation of `link` and `bytes`

Most types encode straightforwardly in JSON; only `link` and `bytes` need special treatment. A link is an object with the single key `$link` and the string-encoded CID as value. The spec's example, a node with a single field `"exampleLink"`:

```
{
  "exampleLink": {
    "$link": "bafyreidfayvfuwqa7qlnopdjiqrxzs6blmoeu4rujcjtnci5beludirz2a"
  }
}
```

Bytes are an object with the single key `$bytes` and a base64-encoded string value (RFC-4648 section 4 "simple" base64: not URL-safe, padding optional):

```
{
  "exampleBytes": {
    "$bytes": "nFERjvLLiw9qm45JrqH9QTzyC2Lu1Xb4ne6+sBrCzI0"
  }
}
```

(DAG-JSON would use `/` as the key name instead of `$link`/`$bytes`; atproto chose the `$` forms as more readable and less confusing for developers.)

Blessed CID formats

The IPFS CID specification is very flexible; atproto restricts it to maximize interoperability:

> The blessed formats for CIDs in atproto are:
> - CIDv1
> - multibase: binary serialization within DAG-CBOR `link` fields, and `base32` for string encoding
> - multicodec: `dag-cbor` (0x71) for links to data objects, and `raw` (0x55) for links to blobs

> - multihash: `sha-256` with 256 bits (0x12) is preferred

AT Protocol specification, "Data Model"

The multicodec split is the part to keep straight: structured data objects (records, commits, MST nodes) are linked with `dag-cbor` (0x71), and arbitrary file bytes (blobs) are linked with `raw` (0x55). SHA-256 is a stable requirement in some contexts (such as repository MST nodes), while blob hash types may evolve. A CID can appear in atproto data three ways: as a `cid-link` field (binary CID with CBOR tag 42 in DAG-CBOR; `$link` object in JSON), as a `string` field with format `cid`, or as a `string` field with format `uri` using the `ipld://` scheme.

Lexicon

Lexicon is a schema definition language used to describe atproto records, HTTP endpoints (XRPC), and event stream messages. It builds on the data model, and is similar in spirit to JSON Schema and OpenAPI, with atproto-specific features and semantics. The current language version is 1.

Lexicon files

Lexicons are JSON files associated with a single NSID. A file is an object with:

- `lexicon` (integer, required): language version, fixed at `1`
- `id` (string, required): the NSID of the Lexicon
- `description` (string, optional)
- `defs` (map of strings-to-objects, required): the definitions, each with a distinct name; a Lexicon with zero definitions is invalid

A file may contain at most one definition of a "primary" type, which should be named `main`. The primary types are:

- `query`: an XRPC Query (HTTP GET)
- `procedure`: an XRPC Procedure (HTTP POST)
- `subscription`: an Event Stream (WebSocket)
- `record`: an object that can be stored in a repository record; its schema includes a `key` field naming the Record Key type, and a `record` field holding an `object` schema

References to definitions within a Lexicon use fragment syntax like `com.example.defs#someView`. A `main` definition is referenced with just the NSID, and in `$type` fields in data this is mandatory: `com.example.record`, never `com.example.record#main`.

Other field types include the concrete types from the data model (`null`, `boolean`, `integer`, `string`, `bytes`, `cid-link`, `blob`), containers (`array`, `object`, `params`), and meta types (`token`, `ref`, `union`, `unknown`). Unions are "open" by default, meaning future schema revisions may add variants and validators should be permissive; a union can also be marked `closed`. All union variants must be

objects including a `$type` field. String fields can carry a `format` constraint tying them to identifier syntaxes covered in this lesson: `did`, `handle`, `at-identifier`, `nsid`, `tid`, `record-key`, `at-uri`, `cid`, plus `datetime`, `uri`, and `language`.

When to use `$type`

A data object needs a `$type` field whenever there could be ambiguity about its content type during validation. The specific rules: `record` objects must always include `$type` (records can travel outside repositories and must be self-describing), and `union` variants must always include `$type`, except at the top level of `subscription` messages. `blob` objects always include `$type`, enabling generic processing.

Lexicon evolution

> Lexicons are allowed to change over time, within some bounds to ensure both forwards and backwards compatibility. The basic principle is that all old data must still be valid under the updated Lexicon, and new data must be valid under the old Lexicon.
>
> - Any new fields must be optional
> - Non-optional fields can not be removed. A best practice is to retain all fields in the Lexicon and mark them as deprecated if they are no longer used.
> - Types can not change
> - Fields can not be renamed
>
> If larger breaking changes are necessary, a new Lexicon name must be used.

AT Protocol specification, "Lexicon"

There is no formal moment when a Lexicon becomes set in stone. At a minimum, public adoption and implementation by a third party, even without explicit permission, indicates it has been released and should not break compatibility. A best practice is to flag experimental status in the name itself, as in `com.corp.experimental.newRecord`.

Validation behavior follows from the evolution rules: data that fails Lexicon validation should be treated as entirely invalid (no partial processing or repair), but *unexpected fields* in otherwise-conforming data must be ignored (at worst, warnings) so that out-of-date implementations tolerate schema evolution. Implementations that deserialize into fixed structures should beware of "clobbering", that is, silently stripping unknown fields when re-serializing.

Authority, publication, and resolution

The authority for a Lexicon is determined by its NSID, rooted in DNS control of the domain authority. That authority has ultimate control of the definition. Lexicon schemas are published publicly as records in atproto repositories using the `com.atproto.lexicon.schema` type, where the record key

is the NSID of the schema. The link from NSID to a specific repository (DID) is a DNS TXT record on the `_lexicon` name, similar to (but distinct from) handle resolution's `_atproto` record.

Authority is granted at the "group" level: all NSIDs differing only in the final name segment are published in the same repository. The spec's worked example:

```
NSID:              edu.university.dept.lab.blogging.getBlogPost
"name":            getBlogPost
authority domain:  blogging.lab.dept.university.edu
DNS TXT name:      _lexicon.blogging.lab.dept.university.edu
DNS TXT value:     did=did:plc:ewvi7nxzyoun6zhxrhs64oiz
record AT-URI:     at://did:plc:ewvi7nxzyoun6zhxrhs64oiz/com.atproto.lexicon.schema/ed
```

A resolving service starts with the NSID, does DNS TXT resolution for `_lexicon.blogging.lab.dept.university.edu`, resolves the resulting DID, finds the PDS, and fetches the record. Resolution is strictly non-hierarchical:

> Lexicon resolution of NSIDs is not hierarchical: DNS TXT records must be created for each authority section, and resolvers should not recurse up or down the DNS hierarchy looking for TXT records.

AT Protocol specification, "Lexicon"

> If the DNS TXT resolution for `_lexicon.blogging.lab.dept.university.edu` failed, the resolving service would *NOT* try `_lexicon.lab.dept.university.edu` or `_lexicon.getBlogPost.blogging.lab.dept.university.edu` or `_lexicon.university.edu`, or any other domain name. The Lexicon resolution would simply fail.

AT Protocol specification, "Lexicon"

An NSID like `edu.university.dept.lab.blogging.getBlogComments` shares the same authority name and must be published in the same repository (with a different record key); `edu.university.dept.lab.gallery.photo` would need its own TXT record (`_lexicon.gallery.lab.dept.university.edu`), which may point at the same or a different repository. A simpler example: `app.toy.record` resolves via `_lexicon.toy.app`. One operational wrinkle: Lexicon record operations are broadcast over the firehose, but DNS resolution changes are not, so resolving services should not cache DNS results for long periods.

Repositories & the MST

Public atproto content (**records**) is stored in per-account repositories ("repos"). All currently active records live in the repository; contents are publicly available, and both content and account deletion are fully supported. The repository is content-addressed (a Merkle tree): every mutation produces a

new commit `data` hash (CID), and commits are cryptographically signed with rotatable signing keys, allowing recursive validation of content in whole or in part.

Repositories are canonically stored as binary DAG-CBOR, a graph of objects referencing each other by CID links. Large binary blobs are *not* stored in the repository; they are referenced by CID. The authoritative location of an account's repository is its PDS, as indicated in the DID document. Repositories can be exported as CAR files for backup or account migration.

Repo structure and paths (v3)

At a high level, a repository is a key/value mapping where keys are path strings and values are records (DAG-CBOR objects). Repo paths currently have a fixed structure:

```
<collection>/<record-key>
```

That is a valid, normalized NSID, a `/`, and a valid record key: no leading slash, always exactly two segments. Because paths for all records in a collection sort together, enumeration and export by collection are efficient. The TID record key scheme was intentionally chosen so that keys within a collection sort chronologically (when TID generation was done correctly, which cannot be relied on in general), and appends are more efficient than random insertions.

Commit objects

The top-level object in a repository is a signed commit, with fields:

- `did` (string, required): the account DID, strictly normalized
- `version` (integer, required): fixed value `3`
- `data` (CID link, required): pointer to the top of the MST
- `rev` (string, TID format, required): repo revision, used as a logical clock; must increase monotonically. Recommended to be the current timestamp as a TID; `rev` values in the "future" (beyond a fudge factor) should be ignored
- `prev` (CID link, nullable): pointer to a previous commit; in v3 the field must exist in the CBOR object but is virtually always `null`
- `sig` (byte array, required): the commit signature, raw bytes

Signing works exactly the way the data-model section implies: populate all fields of an UnsignedCommit (everything except `sig`), serialize with DAG-CBOR, hash the bytes with SHA-256, and sign the binary hash with the account's current signing key (the `#atproto` key from the DID document). The commit's own CID is computed by serializing the *signed* commit as DAG-CBOR. Neither the signature nor the commit records which key or curve was used; that comes from the DID document. So after key rotation, older commit signatures can become ambiguous. The most recent commit should always verify against the current DID document, which implies creating a new commit whenever the signing key rotates.

MST structure

The key/value mapping is stored in a **Merkle Search Tree** (MST): a content-addressed, deterministic data structure holding data in key-sorted order, reasonably efficient for key lookups, range scans, and appends. MST keys are byte arrays (repo path strings encoded as UTF-8); values are CID links to records. The defining property:

> The overall structure and shape of the MST is deterministic based on the current key/value content, regardless of the history of insertions and deletions that lead to the current contents.

AT Protocol specification, "Repository"

Everything else in the repository design leans on this property. The MST is fully reproducible from the bytestring-to-CID mapping, with an exactly reproducible root CID (the `data` field in the commit). Two independent implementations holding the same records will compute the same root hash.

Every node contains key/CID entries plus links to sub-tree nodes, all in key-sorted order (left = lexically first). Each key has a **depth** derived from the key itself, which determines its level in the tree; the top node holds the keys with the highest depth. For the atproto MST, the hash algorithm is SHA-256, counting "prefix zeros" in 2-bit chunks, giving a fanout of 4. To compute a key's depth:

- hash the key (a byte array) with SHA-256, with binary output
- count the number of leading binary zeros in the hash, and divide by two, rounding down
- the resulting positive integer is the depth of the key

The spec's examples, with the given ASCII strings mapping to byte arrays:

```
2653ae71: depth "0"
blue: depth "1"
app.bsky.feed.post/454397e440ec: depth "4"
app.bsky.feed.post/9adeb165882c: depth "8"
```

Nothing about depth is random or history-dependent. It is a pure function of the key, which is exactly why the tree shape is reproducible. An empty repository is a single MST node with an empty entries array (the only situation where an empty leaf is allowed); empty nodes must be pruned from the top and bottom of the tree, but empty *intermediate* nodes are kept so sub-tree links never skip a depth level.

For storage efficiency, keys within a node are prefix-compressed: each entry records how many bytes it shares with the previous key. The first entry in a node carries the full key with prefix length 0, and compression never extends across nodes. This compaction scheme is mandatory, to keep the structure deterministic across implementations. The node schema:

- `l` ("left", CID link, nullable): link to a sub-tree node at a lower level whose keys all sort before the keys at this node

- `e` ("entries", array of objects, required): ordered list of TreeEntry objects:
  - `p` ("prefixlen", integer, required): count of bytes shared with the previous TreeEntry in this node
  - `k` ("keysuffix", byte array, required): remainder of the key after "prefixlen" bytes are removed
  - `v` ("value", CID link, required): link to the record data (CBOR) for this entry
  - `t` ("tree", CID link, nullable): link to a sub-tree node at a lower level whose keys sort after this entry's key but before the next entry's key in this node

When parsing MSTs, and especially when parsing untrusted input, verify key depth and sort order every time, and bound node sizes. Accounts control their own record keys, so a hostile account can mine keys with chosen depths to produce pathological tree shapes (storage overhead and network amplification). Implementations should limit TreeEntries per node to a statistically unlikely maximum.

CID formats in the repository

The blessed CID format for links to commit objects, MST nodes, and records is CIDv1, binary multibase within DAG-CBOR (`base32` in JSON mappings), multicodec `dag-cbor` (0x71), multihash `sha-256` (0x12). These constraints are strictly enforced for structural links: commits with non-compliant `prev` or `data` links are invalid, and MST nodes with non-compliant links to other MST nodes make the entire MST invalid. More flexibility is tolerated when *processing* leaf links from MST to records, in which case implementations should retain those exact CIDs rather than normalizing. When generating new record links, follow the blessed format strictly.

CAR file serialization

Repositories are exported as CAR v1 (Content Addressable aRchive) files, with suffix `.car` and mimetype `application/vnd.ipld.car`. A CAR file has a small header indicating one or more "root" CIDs, followed by a series of binary blocks. In atproto, the first element of the `roots` array must be the CID of the most relevant commit object (for a generic export, the current commit); full exports must include the complete repo structure for that commit, meaning all records and all MST nodes. Block order is not defined, duplicate blocks may appear, and importers should tolerate dangling CID references (links to blobs, for example, which are not stored in the repo) while verifying the completeness of the repository structure itself.

You should now be able to explain

- The two blessed DID methods (`did:web`, `did:plc`), DID syntax rules, and why unsupported methods should still be handled gracefully
- What atproto extracts from a DID document: the handle in `alsoKnownAs`, the `#atproto` Multikey signing key, and the `#atproto_pds` service endpoint
- Why the handle-to-DID link must be verified bidirectionally, and what `handle.invalid` means

- Handle syntax rules and the two resolution methods, the `_atproto` DNS TXT record (`did=…`) and the HTTPS `/.well-known/atproto-did` endpoint, including which result wins on conflict
- NSID structure (reversed domain authority plus name), and which part is case-insensitive versus case-sensitive
- TID structure (64-bit: zero top bit, 53 bits of microseconds, 10-bit clock identifier; 13-char base32-sortable) and why TID global uniqueness cannot be guaranteed
- The four record key types (`tid`, `nsid`, `literal:<value>`, `any`), record key syntax, and why only `(did, collection, rkey)` is a unique tuple
- AT URI structure, why the authority is an identity rather than a network location, and how to build a strong reference (DID-based AT URI plus CID)
- The dual JSON/CBOR data model, why DAG-CBOR is the representation used for signing and hashing, and the `$link`/`$bytes`/`$type` JSON conventions
- The blessed CID formats, including `dag-cbor` (0x71) for data objects versus `raw` (0x55) for blobs
- Lexicon file structure, primary types, `$type` rules, and the four evolution rules that keep schemas forward- and backward-compatible
- How Lexicons are published (`com.atproto.lexicon.schema` records), how they resolve via `_lexicon` DNS TXT records, and why resolution never recurses up or down the DNS hierarchy
- Repository structure: `<collection>/<record-key>` paths, signed commit objects and how they are signed, and CAR file exports
- Why the MST's shape is deterministic from its current contents, how key depth is computed (SHA-256 leading zeros in 2-bit chunks, fanout 4), and the MST node schema